

Configuration du cache et du Load Balancer

1. Cache Redis (FastAPI + fastapi-cache2)

- **Bibliothèque utilisée** : `fastapi-cache2`
- **Backend** : Redis, lancé dans un conteneur Docker
- **Endpoints mis en cache** :
 - `GET /reporting/dashboard`
 - `GET /restock/stock_par_magasin`
 - `POST /stock/create` (invalide le cache associé)
 - `PUT /stock/update` (invalide le cache associé)
- **Durée de vie des caches (TTL)** : 60 secondes pour les appels de lecture
- **Bénéfices** :
 - Réduction de la latence des appels `GET`
 - Diminution des requêtes à la base de données
 - Amélioration globale des performances et de la scalabilité

Exemple de code d'initialisation :

```
from fastapi_cache2 import FastAPICache
from fastapi_cache2.backends.redis import RedisBackend
import redis.asyncio as redis

@app.on_event("startup")
async def startup():
    redis_client = redis.Redis(host="redis", port=6379, db=0)
    FastAPICache.init(RedisBackend(redis_client), prefix="fastapi-cache")
```

2. Load Balancer NGINX

- **Stratégie de répartition utilisée** : Round Robin ou Least Connections (selon le test)
- **Nombre d'instances FastAPI** : 3 conteneurs exposés sur les ports 8000, 8001, 8002
- **Objectif** : Répartir équitablement les requêtes entrantes vers les services backend
- **Résilience** : Les instances défaillantes sont ignorées automatiquement par NGINX si configuré avec `fail_timeout` ou via une solution de supervision externe.

Exemple de configuration NGINX (Round Robin) :

```
upstream fastapi_app {
    server fastapi1:8000;
    server fastapi2:8001;
    server fastapi3:8002;
}

server {
    listen 80;
```

```
location / {
    proxy_pass http://fastapi_app;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}
```

Alternative : stratégie **least_conn** :

```
upstream fastapi_app {
    least_conn;
    server fastapi1:8000;
    server fastapi2:8001;
    server fastapi3:8002;
}
```

Résumé

Composant	Fonction principale	Bénéfices clés
Redis Cache	Mise en cache applicative	Réduit la charge sur la base de données
NGINX	Répartition de charge entre instances FastAPI	Scalabilité horizontale, tolérance aux pannes